



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Московский государственный технический университет имени
Н.Э. Баумана

(национальный исследовательский университет)»

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2 ПО ДИСЦИПЛИНЕ: ТИПЫ И СТРУКТУРЫ ДАННЫХ

«Записи с вариантами, обработка таблиц»

Студент: Колоколов Г.И.

Группа: ИУ7-35Б

Название предприятия **НУК ИУ МГТУ им. Н. Э. Баумана**

Студент _____ Колоколов Г. И.

Преподаватель _____ Никульшина Т. А.

Цель работы	3
Постановка задачи	3
Техническое задание	3
Входные данные	3
Выходные данные	4
Ограничения программы	4
Задача программы	4
Описание структур данных	5
Описание алгоритма	6
Основные функции программы	6
Сравнение используемой памяти и времени работы программы.....	8
Выводы по проделанной работе	9
Контрольные вопросы	9

Цель работы

Приобрести навыки работы с типом данных «запись», содержащим вариантную часть, и с данными, хранящимися в таблицах. Оценить относительную эффективность программы (в процентах) по времени и по используемому объему памяти в зависимости от используемого алгоритма и от объема сортируемой информации.

Постановка задачи

Составить программу для взаимодействия с таблицей литературы, включающую в себя технические, художественные и детские книги. Таблица должна поддерживать сортировку таблицы записей и ключей, добавление записи в конец, поиск и удаление записи по ключу.

Техническое задание

Создать таблицу, содержащую не менее 40-ка записей (тип – запись с вариантами (объединениями)). Упорядочить данные в ней по возрастанию ключей, двумя алгоритмами сортировки, где ключ – любое невариантное поле (по выбору программиста), используя: а) саму таблицу, б) массив ключей. Возможность добавления и удаления записей в ручном режиме, просмотр таблицы, просмотр таблицы в порядке расположения таблицы ключей обязательна. Осуществить поиск информации по варианту.

Вид записи:

Фамилию автора

Название книги

Издательство, количество страниц

Вид литературы:

1. Техническая:

а. О отрасль

б. Отечественная/переводная

с. Год издания

2. Художественная:

а. Тип литературы (роман, пьеса, поэзия)

3. Детская

а. Минимальный возраст

б. Тип детской литературы (стихи, сказки)

Входные данные

1. Имя файла с данными - бинарный файл с расширением *.tbl, содержащий в себе записи с книгами. Размер файла должен быть кратен размеру одной записи.
2. Целое число(от 0 до 13 включительно) - индекс команды, которая будет исполнена программой(см. раздел Реализованные возможности программы)

3. Дополнительный ввод - совокупность численных и текстовых данных, требуемых программой для ввода новой записи, удаления или поиска записи в таблице.

Выходные данные

1. Записи из таблицы записей
2. Записи из таблицы ключей
3. Результат поиска романов по автору
4. Результаты сравнения эффективности работы программы
5. Описания ошибок

Ограничения программы

1. Запрещен ввод пустой строки
2. Длина имени не более 40 символов
3. Длина названия книги не более 100 символов
4. Длина издательства не более 80 символов
5. Длина промышленной отрасли не более 40 символов
6. Максимальное количество элементов в таблице не более 500 записей
7. Длина названия файла не более 255 символов
8. Название файла должно быть формата *.tbl

Задача программы

- 1) Вывод назначения и ограничений программы
- 2) Обработка и валидация ввода
- 3) Ввод имя файла с данными для таблицы записей
- 4) Ввод данных для работы программы
- 5.1) Если введенные данные корректны, то выполнить запрошенное действие
- 5.2) Если введенные данные не корректны, вывести сообщение об ошибке

Реализованные возможности программы

0. Добавить запись
1. Удалить запись
2. Найти романы по автору
3. Вывести таблицу ключей
4. Вывести таблицу по ключам
5. Вывести таблицу по записям
6. Сортировка записей пузырьком
7. Сортировка ключей пузырьком
8. Сортировка записей быстрой сортировкой
9. Сортировка ключей быстрой сортировкой
10. Импортировать данные из файла
11. Экспортировать данные в файл
12. Собрать данные эффективности программы
13. Выйти из программы

Код завершения программы

Код завершения	Символьное обозначение	Описание кода
0	SUCCESS	Успешное завершение программы
1	NULLPTR_ERROR	В функцию был передан нулевой указатель
2	OPEN_FILE_ERROR	Ошибка при открытии файла
3	WRONG_FILENAME_ERROR	Ошибка формата имени файла
4	WRONG_FILE_FMT_ERROR	Формат файла не соответствует заданным в программе записям
5	READ_ERROR	Ошибка при считывании данных
6	ADD_OVERFLOW	Ошибка при добавлении записи в полностью заполненную таблицу
7	NAN_ERROR	Ошибка при считывании числа
8	NOT_FOUND	Запись не найдена
9	EMPTY_TABLE	Таблица не содержит записей

Описание структур данных books.h

```
// Описывает жанр книги
typedef enum
{
    TECHNICAL,
    FICTION,
    CHILDREN
} EBookGenre;

// Описывает жанр художественной и детской литературы
typedef enum
{
    NOVEL,
    PLAY,
    POETRY,
    FAIRY_TALE,
    CHILD_POETRY
} EBookType;

// Структура для технической литературы
typedef struct
{
    char industry[MAX_INDUSTRY_LEN];
    bool domestic;
    size_t publishYear;
} TechnicalBook;
```

```

// Структура для художественной литературы
typedef struct
{
    EBookType type;
} FictionBook;

// Структура для детской литературы
typedef struct
{
    EBookType type;
    size_t minimalAge;
} ChildrenBook;

// Объединение для хранения вариативной части записи
typedef union
{
    TechnicalBook technical;
    FictionBook fiction;
    ChildrenBook children;
} UBookData;

// Структура целиком описывающая книгу(невариативная + вариативная части)
typedef struct
{
    char authorSurname[MAX_SURNAME_LEN + 1];
    char bookTitle[MAX_TITLE_LEN + 1];
    char publisher[MAX_PUBLISHER_LEN + 1];
    size_t pageCount;
    EBookGenre genre;
    UBookData data;
} Book;

```

table.h

```

// Структура для хранения ключа и индекса в массиве записей
typedef struct
{
    char authorSurname[MAX_SURNAME_LEN];
    size_t index;
} Key;

// Структура для хранения записей(таблица)
typedef struct
{
    size_t size;
    Key keySet[MAX_ENTRIES];
    Book entrySet[MAX_ENTRIES];
} Table;

```

Описание алгоритма

1. Программа выводит правила и ограничения
2. Программа выводит меню для пользователя
3. Программа будет продолжать предлагать пользователю команды до тех пор, пока он выберет пункт завершения программы

Основные функции программы

Функции для работы с таблицей

```

// добавление записи в таблицу
int add_entry(Table *table, Book *entry);

// удаление записи из таблицы по названию
int remove_first_by_title(Table *table, char *title);

// Поиск всех романов автора

```

```

int find_all_novels_by_author(Table *table, char *author);

// Вывод таблицы ключей
int print_key_array(Table *table);

// Вывод таблицы записей по таблице ключей
int print_table_by_keys(Table *table);

// Вывод таблицы записей
int print_table_by_entries(Table *table);

```

Функции для сортировки таблицы

```

// Сравнение ключей таблицы
int compare_keys(Key *first, Key *second);

// Сравнение записей таблицы
int compare_entries(Book *first, Book *second);

// Поменять ключи местами
void swap_keys(Key *first, Key *second);

// Поменять записи местами
void swap_entries(Book *first, Book *second);

// Пузырьковая сортировка таблицы ключей
void bsort_table_by_keys(Table *table);

// Пузырьковая сортировка таблицы записей
void bsort_table_by_entries(Table *table);

// Быстрая сортировка по таблице ключей
void qsort_table_by_keys(Table *table);

// Быстрая сортировка по таблице записей
void qsort_table_by_entries(Table *table);

```

Функции для сбора эффективности программы

```

// Количество сборов времени сортировки для расчета среднего
#define TRIES 50

// Функция для нахождения эффективности для сортировки пузырьком
void get_time_efficiency_bsort(Table *table, FILE *file, size_t size);

// Функция для нахождения эффективности для сортировки пузырьком
void get_time_efficiency_qsort(Table *table, FILE *file, size_t size);

// Функция для нахождения эффективности таблицы ключей относительно таблицы записей
void get_memory_efficiency(void);

// Собирает данные об эффективности программы
void get_program_efficiency(void);

```

Функции для взаимодействия с пользователем

```

// Основная функция для работы программы, которая выводит меню
// и запрашивает у пользователя индекс команды для исполнения
int execute_operation(Table *table);

// Ввод строки
void input_string(char *field, char *title, size_t max_size);

// Ввод перечисления
size_t input_enum(size_t max_options, char **options);

// Ввод числового значения с возможность задания диапазона числа

```

```
size_t input_value(char *title, bool min_limit, bool max_limit, int min_value,
int max_value);

// Валидация имени файла с таблицей
int validate_filename(char *filename);

// получение сообщение об ошибке по коду ошибки
char *get_error_message(int error);
```

Сравнение используемой памяти и времени работы программы

Замеры времени происходят путем запуска сортировки несколько раз(50 в моей программе) и взятия среднего от времени всех запусков.

Время для сортировки пузырьком (в наносекундах)

Количество элементов	Таблица ключей	Таблица элементов	Быстрота сортировки ключей относительно записей
50	31168	78105	60%
100	49767	123636	60%
150	112661	278470	60%
200	202351	497207	59%
250	320496	787737	59%
300	470540	1153287	59%
350	619525	1520176	59%
400	809240	1985933	59%
450	1018101	2503817	59%
500	1273464	3136673	59%

Время для быстрой сортировки (в наносекундах)

Количество элементов	Таблица ключей	Таблица элементов	Быстрота сортировки ключей относительно записей
50	2717	3282	17%
100	6438	7171	10%
150	10311	11738	12%
200	14253	15653	9%
250	19137	20337	6%
300	23469	25455	8%
350	27336	30848	11%
400	31828	35277	10%
450	38661	41295	6%
500	42593	45286	6%

Память: запись ключа занимает 49 байт (41 символов и `size_t`), запись в таблице занимает 296 байт
49/296 ~ на 16% памяти больше

Время: как можно заметить, в среднем, чем больше элементов, тем эффективнее сортируется таблица ключей относительно таблицы записей

Выводы по проделанной работе

Была написана такая структура как таблица, с реализованными операциями удаления, добавления и сортировки. Удалось сравнить насколько эффективна таблица ключей по скорости относительно сортировки таблицы записей. Так было достигнуто значительное ускорение работы таблицы при добавлении таблицы ключей (~60% для сортировки пузырьком и ~6-8% для быстрой сортировки). Такие результаты сортировки были достигнуты из-за того, что в таблице использовалась строка в качестве ключа, что уменьшает эффективность программы, так как сравнение строк достаточно медленная операция.

Контрольные вопросы

1. Как выделяется память под вариантную часть записи?

Объем памяти, необходимый для записи с вариантами складывается из длин полей фиксированной части и максимального по длине поля вариантной части. Таким образом на одну запись тратится меньше памяти.

2. Что будет, если в вариантную часть ввести данные, несоответствующие описанным?

Так как тип данных в вариантной части при компиляции не проверяется, то вариантной части будет присвоено неверное значение, что скорее всего может привести к ошибке в программе.

3. Кто должен следить за правильностью выполнения операций с вариантной частью записи?

Тип данных в вариантной части при компиляции не проверяется, поэтому, контроль за правильностью ее использования возлагается на программиста.

4. Что представляет собой таблица ключей, зачем она нужна?

Таблица ключей - массив, состоящий из ключа и индекса записи в массиве записей. Таблица ключей нужна для более эффективных операций над ключами таблицы.

5. В каких случаях эффективнее обрабатывать данные в самой таблице, а когда – использовать таблицу ключей?

Таблицу ключей выгодно использовать когда она занимает не так много памяти относительно основной таблицы или когда у таблицы может измениться размер. Когда таблицы имеет статичный размер (например 50 элементов) сильного выигрыша по скорости не будет, но при этом будет задействована

дополнительная память, в таком случае будет эффективнее обрабатывать данные в самой таблице.

6. Какие способы сортировки предпочтительнее для обработки таблиц и почему?

Для обработки таблиц предпочтительнее сортировки, которые делают меньше перестановок, так как записи могут быть достаточно большими по объему и их копирование будет уменьшать эффективность программы.